

Guía: construir el Ahorcado con ATDD usando IA

Esta guía es el **procedimiento** que el **grupo** tiene que seguir. Lo que se evalúa no es el código: es que sigan el proceso y entiendan cada decisión.

Es un **trabajo de equipo**: escriben los tests **por turnos, rotando** quién hace cada uno (ver §3, regla 7) y trabajan juntos en cada paso. En las instrucciones, cuando decimos "quien está al teclado" nos referimos al integrante que arrancó ese test en su turno; las decisiones, en cambio, las toma el grupo.

El código de producción lo pueden escribir **ustedes mismos** o pedirselo a la **IA** — es su elección, y pueden mezclar (codear unas partes, pedir otras). El proceso es **idéntico** en los dos casos. La IA es una herramienta opcional, no el centro del ejercicio.

Regla de oro: **el grupo navega**. Ustedes deciden cada paso (qué test sigue, qué nombre tiene, cuándo refactorizar). Si alguien no puede explicar por qué existe un test o qué línea lo hace pasar, frenen — da igual si esa línea la escribió un integrante o la IA.

1. Qué vas a construir

Un juego del Ahorcado con **la lógica separada de la interfaz**, manejado con **dos loops de tests**:

- **UI** (la interfaz): la pantalla con la que juega el usuario. Lee teclas/clicks y muestra el estado. **No** contiene lógica de juego: para todo lo de negocio le pregunta al objeto `Ahorcado`.
- **Ahorcado** (la lógica): un objeto TypeScript en memoria que no sabe nada de pantallas, DOM ni `console.log`. Recibe acciones y devuelve estado. Es lo que la UI consume.

No hay REST ni base de datos. Todo en memoria, en TypeScript.

```
Navegador → index.html → arranque (index.ts) → UI (main.ts) → Ahorcado
                                lee ?word=                sin lógica      lógica, sin DOM
```

2. El doble loop (modelo mental)

El loop **externo** es un **Acceptance Test en Cucumber/Gherkin** que corre contra la **app real en un navegador** (Playwright como driver): describís el comportamiento en `Given/When/Then` y verificás lo que ve el usuario. El loop **interno** es sobre el objeto `Ahorcado` (Vitest): lógica pura, sin DOM.

```

LOOP EXTERNO (ATDD = Cucumber/Gherkin, app REAL en el browser)
1. Escribís un Acceptance Test (.feature) que abre la página,
   actúa como usuario (tipea una letra) y verifica lo que se VE
   → ROJO (rojo honesto: si la app no arranca o no funciona, FALLA)

  LOOP INTERNO (UT con Vitest, sobre el objeto Ahorcado)
  a. Escribís UN unit test sobre `Ahorcado`
  b. Lo ves ROJO
  c. Escribís (o pedís) el MÍNIMO código → VERDE
  d. Refactor (si hace falta), sigue VERDE
  e. ¿Falta lógica para el AT? volvé a (a)

2. Cableás la UI + el arranque (index.html) al objeto Ahorcado
3. El AT pasa a VERDE → siguiente AT

```

- **Loop externo (AT Cucumber):** prueba la rebanada vertical **completa** — navegador + arranque + UI + dominio. Queda rojo hasta que la app realmente funciona.
- **Loop interno (UT):** maneja la lógica de `Ahorcado` en ciclos rojo→verde→refactor, en milisegundos, sin tocar el navegador.

Por qué el AT corre contra la app real y no contra un test de UI en memoria (jsdom): un test que monta solo un componente puede dar **verde aunque la app real ni arranque** (nunca probó el `index.html` ni el arranque). El AT corre contra la app levantada de verdad, así que su verde **significa que funciona**. Es la diferencia entre "el componente renderiza" y "el juego funciona".

El AT ejercita el objeto `Ahorcado` **real** dentro de la app real (no un mock).

2.1 El camino no es único

En una kata de TDD el próximo test está casi forzado por el anterior: el espacio de decisión es angosto. Con **dos loops** aparece un eje nuevo. Además de *qué* test sigue, ustedes deciden **en qué nivel lo escriben** y **cuánto se bancan una solución falsa arriba antes de bajar al dominio**. Eso no lo resuelve la técnica: lo resuelve el criterio de quien está diseñando, test a test.

Las dos posturas extremas son ambas legítimas:

- **Bajar enseguida.** Apenas el AT está rojo, se va al loop interno y se diseña `Ahorcado` desde el comportamiento que el AT pide (la escuela mockista lleva esto al límite: dobles para descubrir la interfaz, ver §8).
- **Bancar el AT rojo.** Se resuelve provisoriamente en la UI —incluso con valores hardcoded— y el dominio aparece recién cuando la presión de diseño se hace sentir, en un **refactor**.

El segundo camino tiene una ventaja didáctica: hace **visible** el momento en que el objeto se justifica, en vez de darlo por sentado. Y tiene un riesgo: si te quedás arriba demasiado tiempo, acumulás lógica en la pantalla que después es cara de mover.

Tres señales de que llegó la hora de bajar a `Ahorcado`:

1. Necesitás **estado entre interacciones** (qué letras se intentaron, cuántas vidas quedan).
2. La lógica quedó en un lugar donde **no la podés unit-testear**, porque para ejecutarla hace falta un navegador.
3. Apareció un **segundo caso** y sostener el fake ya cuesta más que escribir la implementación real.

El camino es libre; la disciplina no. Elijan la ruta que quieran, pero el rojo va antes que el verde, el código es el mínimo, y el AT corre contra la app real. En la defensa no se evalúa *qué* camino tomaron —no hay uno correcto— sino si pueden sostener **por qué** tomaron ése.

3. Reglas del proceso (no negociables)

Estas reglas valen **codees vos o le pidas a la IA**. Son la disciplina del ejercicio.

1. **No test, no code.** No se escribe código de producción si no hay un test fallando que lo justifique.
2. **Un paso a la vez.** Hacé el paso actual y nada más. No te adelantes a resolver varios ATs juntos.
3. **Ves el rojo antes del verde.** Antes de correr un test, *predecí* si va a estar rojo o verde y por qué. Después corré y verificá.
4. **Mínimo código.** Solo lo necesario para pasar el test actual. Nada que otro test no exija.
5. **Vos nombrás los tests.** El nombre del test es una decisión de diseño tuya (la tomás vos aunque el código lo escriba la IA).
6. **Rojo y verde como commits separados; integrás solo en verde.** Por cada test, un commit con el test fallando (**rojo**) y otro con el código que pasa (**verde**), en ese orden. El rojo queda como **ancestro** del verde en el historial: esa es la prueba de "test primero" — **no** hace falta pushear en rojo. A la rama de integración subís **solo cuando está en verde**: nunca rompés el build compartido. Poné el estado en el **mensaje del commit** con un prefijo: **RED:** el commit del test que falla y **GREEN:** el del código que lo pasa (opcional **REFACTOR:**). Así el orden test-primero se ve de un vistazo en `git log --oneLine`. **Commiteá el rojo apenas lo ves, antes de escribir una sola línea de producción:** si te dejás llevar por el impulso y seguís de largo hasta el verde, te comés el commit del rojo y perdés la evidencia de test-primero (reconstruirla después es incómodo y poco honesto).
7. **Rotación de autor.** Cada test nuevo lo arranca un integrante **distinto**, por turnos, y commitea con **su propia identidad** (idealmente su propia cuenta de GitHub). El historial muestra que trabajaron todos y que nadie acaparó.

Y la integración continua queda verde. Pusheás **solo cuando estás en verde**: commiteás el rojo y el verde localmente y subís con el **verde como tope**. CI corre sobre ese tope, así que el build compartido **nunca queda roto**. El commit rojo viaja en el historial como ancestro del verde —ahí está la evidencia— pero nunca es lo que CI evalúa.

Si codeás vos

Aplicás las reglas a mano: escribís el test, lo ves rojo, escribís el mínimo código, verde, refactor, commit. Tener que escribirlo te obliga a entenderlo — es la forma más sólida de aprender el ciclo.

Si usás la IA

La IA teclea, pero las reglas no cambian. Pedidos acotados, un paso por vez. Si la IA te devuelve **más** que el paso actual, **revertilo** y pediselo de nuevo más chico.

Prompts que sí podés usar:

- "Escribí solo el acceptance test en Cucumber/Gherkin (un `.feature` + sus steps) que actúe como usuario sobre la app real para: . No implementes la UI ni la lógica todavía."
- "Esta es nuestra historia con sus criterios de aceptación. Proponé los escenarios Gherkin que la confirman, uno por criterio, sin steps ni código. ¿Hay algún criterio que no se pueda escribir como escenario?" (los escenarios los revisa y los elige el grupo; la IA propone)
- "Escribí solo el unit test del objeto `Ahorcado` para: . No escribas código de producción todavía."
- "Acá está el test fallando. Implementá el mínimo código para que pase. No agregues nada que otro test no exija."
- "¿Hay algún refactor seguro acá sin cambiar comportamiento? Si no, decí 'no'."

Prompts prohibidos (rompen el proceso):

- ❌ "Hacé el juego del ahorcado completo."
- ❌ "Implementá todos los tests que se te ocurran."
- ❌ "Resolvé todos los ATs."

4. La escalera de ATs (un peldaño a la vez)

Construís **de a un AT**: no empieces el siguiente hasta que el anterior esté verde. Cada AT es algo que el **usuario ve/hace en la pantalla**. La tabla de abajo es **una escalera posible** (una referencia), no un orden obligatorio.

#	Acceptance Test	Lo que ve/hace el usuario
1	Iniciar partida	Empieza con palabra "GATO" → ve <code>_ _ _ _</code> y 6 vidas
2	Acertar letra	Tipea <code>A</code> → ve <code>_ A _ _</code> , vidas siguen en 6
3	Fallar letra	Tipea <code>E</code> → sigue <code>_ _ _ _</code> , vidas = 5
4	Ganar	Completa todas las letras → ve mensaje "GANASTE"
5	Perder	6 fallos → ve "PERDISTE" y la palabra revelada
6	Letra repetida	Re-tipea una letra ya intentada → no penaliza e informa
7	Entrada inválida	Tipea algo que no es letra, o juega con la partida terminada

Esta guía es una referencia de ATs y funcionalidades, no un camino único. Si vienen trabajando con un **Story Map** del Ahorcado, **pueden seguir el orden de su mapa sin ningún problema** — de hecho es lo natural. Lo que **no** cambia es la disciplina del proceso: un AT a la vez, rojo honesto antes del verde, la lógica en Ahorcado. Lo que eligen libremente es **qué ATs toman y en qué orden**.

Cada Historia de Usuario suele mapear a varios ATs. Una US del Story Map (p. ej. "como jugador quiero adivinar letras para ir descubriendo la palabra") normalmente se descompone en **varios** Acceptance Tests —acertar, fallar, letra repetida, ganar...—: cada AT es un comportamiento concreto y observable, y la US es la sombrilla que los agrupa. Tomen cada historia de su mapa y derívenla en los ATs que haga falta, de a uno.

4.1 Conectar los tests con sus Historias de Usuario (trazabilidad)

Ustedes ya tienen **Historias de Usuario** armadas. Un AT no aparece de la nada: es la **Confirmación** de una historia (la tercera C de *Card, Conversation, Confirmation*) escrita de forma que la máquina la pueda ejecutar. La cadena completa es:

Historia de Usuario	→	Criterios de aceptación	→	Escenarios (AT)	→	Unit Tests
"como jugador quiero adivinar letras..."		"una letra ausente cuesta una vida"		el .feature en rojo/verde		lo que tiene que saber hacer Ahorcado

Trazabilidad es poder recorrer esa cadena **en los dos sentidos**, mirando el repo y sin tener que preguntarle a nadie:

- **De arriba hacia abajo:** "mostrame los tests de la HU-02" → ¿cuáles son? ¿están verdes? ¿entonces la historia está terminada?
- **De abajo hacia arriba:** "¿por qué existe este escenario?" → ¿de qué historia y de qué criterio sale?

Un escenario que no se puede subir hasta ninguna historia es una señal: **o falta una historia, o el test sobra**. Y un criterio de aceptación que no se puede escribir como escenario ("el juego es fácil de usar") es otra: **la historia todavía no está lista** para desarrollarse — no es *testable* (la T de INVEST). Reescribanla o pártanla antes de abrir el .feature.

El mínimo que no se negocia

1. Cada historia tiene un **identificador estable** (HU-01, HU-02 ... o el número de issue si la tienen en un tablero). Si su Story Map no tiene IDs, pónganselos.
2. **Cada AT referencia la historia** (y, si quieren, el criterio) de la que sale.
3. **Cada historia que toman lista sus AT** y se puede saber si está terminada: una historia está *Done* cuando **todos** sus escenarios están verdes.

Cómo lo logran lo deciden ustedes. Eso es lo que tienen que **explorar**.

Mecanismos para explorar

No son excluyentes: la gracia es probar, combinar y quedarse con lo que les sirva.

Mecanismo	Cómo se ve	Qué les da	Qué cuesta
Una Característica por historia	la narrativa <i>Como / quiero / para</i> como descripción del <code>.feature</code> y una <code>Regla:</code> por criterio de aceptación	el <code>.feature</code> se lee como la historia; negocio lo reconoce	¿qué hacen con un escenario que sirve a dos historias?
Etiquetas de Gherkin	<code>@HU-02</code> sobre la <code>Característica</code> , <code>@CA-2</code> sobre un escenario	correr solo una historia: <code>npx playwright test --grep @HU-02</code>	disciplina: una etiqueta mal escrita no rompe nada
Tablero / issues (GitHub Issues o Projects, Jira, Trello...)	la historia es el issue <code>#12</code> ; el commit dice <code>RED: ... (#12)</code> y el último verde <code>closes #12</code>	la historia se cierra sola cuando termina; <code>git log --grep '#12'</code>	depende de una herramienta afuera del repo
Commits	<code>RED: HU-02 fallar letra descuenta una vida</code>	el historial cuenta qué historia se estaba construyendo	nada lo valida: si alguien se olvida, se pierde
Unit Tests	<code>describe("HU-02 ...")</code> y <code>vitest run -t HU-02</code> , o <code>tags</code> de Vitest (<code>tags</code> en la config + <code>--tags-filter HU-02</code>)	bajar hasta el dominio desde la historia	ver la pregunta 3 de abajo
Matriz en NOTES.md	tabla HU → criterio → escenario → UTs → estado	todo en un lugar, legible en la defensa	es a mano: se desactualiza si no la mantienen

Un ejemplo que combina los dos primeros (funciona tal cual con el stack de §6):

```
# features/adivinar-letras.feature
# language: es
@HU-02
Característica: Adivinar letras
  Como jugador
  quiero adivinar letras
  para ir descubriendo la palabra

  Regla: una letra presente se revela sin perder vidas

  @CA-1
  Escenario: El jugador acierta una letra
    Dado una partida con la palabra "GATO"
    Cuando el jugador adivina la letra "A"
    Entonces se ve la palabra "_ A _"
    Y se ven 6 vidas

  Regla: una letra ausente cuesta una vida

  @CA-2
  Escenario: El jugador falla una letra
    Dado una partida con la palabra "GATO"
    Cuando el jugador adivina la letra "E"
    Entonces se ve la palabra "_ _ _"
    Y se ven 5 vidas
```

```
npx bddgen && npx playwright test --grep @HU-02  # los AT de la historia 2
npx bddgen --tags "@HU-02 and @CA-2" && npx playwright test  # un solo criterio
```

Los escenarios **heredan** las etiquetas de su **Característica**: con poner `@HU-02` arriba, todos quedan marcados.

Preguntas que tienen que poder contestar (en la defensa)

No tienen una única respuesta correcta; tienen que tener **la de ustedes**, y sostenerla.

1. **¿Una **Característica** por historia o por comportamiento?** ¿Qué pasa cuando un escenario (p. ej. *ganar*) sirve a más de una historia?
2. **Cuando cambia una historia** (ahora son 8 vidas en vez de 6), ¿cuánto tardan en encontrar **todos** los tests afectados? Pruébenlo.
3. **¿Etiquetan los Unit Tests con la historia?** **Ahorcado** no sabe nada de historias, y la misma lógica (p. ej. revelar letras) sirve a varias. ¿La traza termina en el AT y la lista de UTs de `NOTES.md` hace el último tramo, o baja hasta el UT?
4. **¿Dónde vive la verdad** de qué historias hay y en qué estado están: en el tablero, en los `.feature` o en `NOTES.md`? Si está en dos lugares, ¿cómo evitan que se contradigan?
5. **¿Cómo saben que una historia está **Done**** sin abrir el código? ¿Lo puede ver alguien que no es del grupo?

Lo que se evalúa no es el mecanismo, es la cadena. En la defensa se elige una historia de su mapa y se pide ver sus escenarios y sus UTs, o se elige un test al azar y se pide la historia de la que sale. Si la respuesta está en el repo, cumple.

5. Cómo trabajar UN AT (el ciclo completo)

Tomemos el **AT 2 — Acertar letra** como ejemplo.

Paso 0 — Elegí la historia y el criterio que vas a confirmar

Antes de escribir nada: **¿de qué Historia de Usuario sale este AT, y qué criterio de aceptación confirma?** (en el ejemplo, la historia *adivinar letras* y el criterio *"una letra presente se revela sin perder vidas"*). Si no lo pueden contestar, no arranquen el AT: vuelvan al mapa. La referencia a la historia queda escrita en el `.feature` desde el primer rojo (ver §4.1).

Paso 1 — Escribí el Acceptance Test en Cucumber/Gherkin (y velo rojo)

El AT son dos archivos: un `.feature` que describe el comportamiento en lenguaje de negocio, y los **step definitions** que lo automatizan contra la app **real** en el navegador. El **seam de test** es la URL `?word=GATO`: así la palabra secreta es determinista (en un AT contra la app no podés pasar la palabra por parámetro de función; la inyectás por la URL).

```
# features/acertar-letra.feature
# language: es
Característica: Acertar letra

  Escenario: El jugador acierta una letra
    Dado una partida con la palabra "GATO"
    Cuando el jugador adivina la letra "A"
    Entonces se ve la palabra "_ A _ _"
    Y se ven 6 vidas
```

```
// features/steps/ahorcado.steps.ts
import { expect } from "@playwright/test";
import { createBdd } from "playwright-bdd";

const { Given, When, Then } = createBdd();

Given("una partida con la palabra {string}", async ({ page }, palabra: string) => {
  await page.goto(`/?word=${palabra}`);
});

When("el jugador adivina la letra {string}", async ({ page }, letra: string) => {
  const input = page.getByRole("textbox");
  await input.fill(letra);
  await input.press("Enter");
});

Then("se ve la palabra {string}", async ({ page }, esperada: string) => {
  await expect(page.getByTestId("word")).toHaveText(esperada);
});

Then("se ven {int} vidas", async ({ page }, vidas: number) => {
  await expect(page.getByTestId("lives")).toHaveText(String(vidas));
});
```

Corré `npm run at`. **Predecí:** ¿rojo o verde? (Rojo: el input todavía no existe.) Confirmá. Este rojo es **honesto**: si la app ni siquiera arranca, también falla. Fijate que los steps se **reutilizan** entre features (el `Dado una partida...` ya lo escribiste para el AT1).

Paso 2 — Enumerá los UT del objeto `Ahorcado` (vos, antes de pedirle a la IA)

Para que ese AT pase, la UI le va a preguntar al objeto `Ahorcado`. Esa lógica tiene que cumplir, por ejemplo:

- adivinar una letra presente revela **todas** sus ocurrencias ("ALA" + A → A _ A)
- es case-insensitive (a == A)
- la palabra enmascarada se representa bien (guiones para lo oculto)
- la letra acertada **no** descuenta vidas

Esa lista es un **entregable**. Demuestra que pensaste el slice antes de codear.

Paso 3 — Loop interno: un UT por vez sobre `Ahorcado`

Para cada ítem de la lista:

1. Escribí (o pedile a la IA) **un** unit test sobre `Ahorcado`. Sin DOM.

2. Velo rojo.
3. Escribí (o pedile a la IA) el mínimo código. Velo verde.
4. ¿Refactor? Hacelo, segui verde.
5. Commit.

```
// tests/Ahorcado.test.ts
import { describe, it, expect } from "vitest";
import { Ahorcado } from "../src/domain/Ahorcado";

it("revela todas las ocurrencias de la letra acertada", () => {
  const juego = new Ahorcado("ALA");
  juego.adivinar("A");
  expect(juego.palabraEnmascarada()).toBe("A _ A");
});
```

Paso 4 — Cableá la UI + el arranque y cerrá el AT

Cuando la lógica está verde, sumá a la UI (`main.ts`) lo **mínimo** para que el AT pase: el input, leer la tecla, pedirle a `Ahorcado` que adivine y volver a renderizar. La primera vez también cableás el arranque (`index.ts` + `index.html`). Corré `npm run at`: tiene que estar **verde**.

Paso 5 — Mirá la app funcionando (con tus propios ojos)

Antes de pasar al siguiente AT, **abrí la app y jugá un momento**: levanta `npm run dev` y entrá a `http://localhost:5173/?word=GAT0`. El verde del AT prueba el **comportamiento** que automatizaste, pero el navegador te muestra lo que el AT **no** asegura: que se vea bien, que el layout tenga sentido, que la experiencia no sea rara (una pantalla pelada y sin estilo igual pasa el AT). Mirar la app es **parte del ciclo**, **no un extra**. Recién ahí pasás al AT 3.

6. Estructura del proyecto (separación lógica/UI)

El stack que usamos (y cómo cambiarlo)

Esta guía usa: **TypeScript**, **Vitest** (unit tests del dominio), **Cucumber/Gherkin** para los AT vía **playwright-bdd** con **Playwright** como driver del navegador, y **Vite** (dev server). Es lo que recomendamos por su baja fricción y feedback rápido.

El grupo puede elegir otro stack si lo prefiere. La elección es libre mientras cumpla cuatro cosas:

1. **Lógica separada de la UI** (un objeto/módulo de dominio, sin DOM).
2. Un **runner de unit tests rápido** para el dominio (loop interno).
3. **Cucumber (Gherkin)** para los AT, con un driver que maneje la app **real en un navegador** (Selenium, Playwright o similar) — no un test de UI en memoria.
4. Un **seam** para fijar la palabra secreta de forma determinista en el AT.

Equivalencias por lenguaje (mismo Gherkin, distinto binding): **Reqnroll** (C#), **Cucumber-JVM** (Java/Kotlin), **behave** / **pytest-bdd** (Python), **CucumberJS** / **playwright-bdd** (JS/TS), con Selenium o Playwright para manejar el navegador. **El patrón de dos loops no cambia**; solo cambian las herramientas. El resto de esta sección muestra la estructura para el stack recomendado.

Estructura y setup (stack recomendado)

```
src/
  domain/
    Ahorcado.ts      ← toda la lógica. Sin DOM, sin console, sin I/O.
  ui/
    main.ts          ← UI (mountApp): monta la pantalla, lee el input,
                      muestra estado. Para lo de negocio, usa Ahorcado.
    index.ts          ← arranque (composition root): lee ?word= y monta la app.
  index.html          ← página que carga el arranque.
  tests/
    Ahorcado.test.ts ← unit tests del objeto, con Vitest (muchos, rápidos)
  features/
    *.feature          ← acceptance tests en Gherkin (uno por feature)
  steps/
    *.steps.ts         ← step definitions (automatizan los .feature; se reutilizan)
```

Reglas de dependencia:

- `domain/` **no importa nada** de `ui/` ni del DOM.
- `ui/` **solo** usa métodos públicos de `Ahorcado`. Si la UI tiene un `if` con lógica de juego (¿gané?, ¿la letra está?), eso va en `Ahorcado`.

Setup mínimo:

```
npm init -y
npm i -D vitest @playwright/test playwright-bdd
npx playwright install chromium
```

(Vite viene como dependencia de Vitest, así que `npm run dev` funciona sin instalarlo aparte.) Necesitas **dos configs, tres scripts** y un `.gitignore`.

`vitest.config.ts` (unit tests del dominio, sin DOM):

```
import { defineConfig } from "vitest/config";

export default defineConfig({
  test: {
    environment: "node",
    include: ["tests/**/*.test.ts"],
  },
});
```

`playwright.config.ts` (AT contra la app real; `defineBddConfig` arma el `testDir` a partir de los `.feature` + steps, y el `webServer` levanta la app):

```
import { defineConfig } from "@playwright/test";
import { defineBddConfig } from "playwright-bdd";

const testDir = defineBddConfig({
  features: "features/**/*.feature",
  steps: "features/steps/**/*.ts",
});

export default defineConfig({
  testDir,
  use: { baseURL: "http://localhost:5173" },
  // Si la app no arranca, los AT fallan (rojo honesto).
  webServer: {
    command: "npm run dev",
    url: "http://localhost:5173",
    reuseExistingServer: !process.env.CI,
  },
});
```

Los tres scripts van en `package.json` (5173 es el puerto por defecto de Vite):

```
"scripts": {
  "dev": "vite",
  "test": "vitest",
  "at": "bddgen && playwright test"
}
```

Y un `.gitignore` con lo generado: `node_modules/`, `.features-gen/`, `test-results/`, `playwright-report/`, `coverage/`, `dist/`.

`bddgen` genera, a partir de los `.feature` + `steps`, los tests que corre Playwright (en `.features-gen/`, que conviene ignorar en git).

La primerísima corrida de `npm run at` —cuando todavía no existe `index.html`— no falla con un `assert` sino como **timeout del `webServer` (~60 s)**: Vite levanta, pero como no hay página que responda `200`, Playwright espera el arranque y se rinde. Es **rojo honesto** igual (la app no funciona). En cuanto exista un `index.html` mínimo (con el `<div id="app">`), los rojos siguientes pasan a ser `asserts` limpios y rápidos. Si no querés esa espera en el primer AT, creá ese `index.html` mínimo antes de correrlo.

7. Entregable y evaluación

Qué entregás:

- El repo con el historial de commits (rojo/verde/refactor visible).
- La lista de UT por cada AT (puede ir en un `NOTES.md`).
- La **trazabilidad** Historia de Usuario → AT → UT, con el mecanismo que eligieron (§4.1), y el Story Map / tablero con los IDs de las historias accesible desde el repo.

Cómo se evalúa (defensa oral):

- Elijo un commit al azar y me explicás: qué test agregaste, por qué, y qué línea lo hace pasar.
- Me explicás por qué el AT (Cucumber) corre contra la app real y por qué la lógica vive en `Ahorcado` y no en la UI.
- Elijo una historia de su mapa y me mostrás sus escenarios y sus UTs; o elijo un test y me decís de qué historia sale. Y me contás **por qué** eligieron ese mecanismo de trazabilidad (§4.1).
- No importa si el código lo escribiste vos o la IA: si no podés explicar una línea, ese punto no suma.

Evidencia en git (orden y trabajo en equipo):

- **Orden test-primero:** en el historial, por cada test tiene que verse un commit **RED:** como ancestro de su **GREEN:**. El tope de la rama (lo que corre CI) siempre quedó en verde.
- **Rotación / participación:** miro que los autores roten y que el trabajo esté repartido.

```
git log --oneline                                # ver la alternancia RED: / GREEN:
git log --format='%h %an %ad %s' --date=iso      # autor + fecha + mensaje, en orden
git shortlog -sne                               # cuántos commits hizo cada integrante
```

Checklist de "Done" por AT:

- ☐ El AT sale de una **Historia de Usuario** y de un criterio de aceptación, y esa referencia se ve en el repo (§4.1).
- ☐ El AT (Cucumber/Gherkin) existe, corre contra la app real y pasó por el rojo.
- ☐ Hay UTs sobre `Ahorcado` que cubren la lógica del slice.
- ☐ El AT está verde.
- ☐ **Miraste la app funcionando en el navegador** (`npm run dev`) y se ve/juega bien.
- ☐ La UI no tiene lógica de juego (toda en `Ahorcado`).
- ☐ Hiciste refactor (o justificás por qué no hacía falta).
- ☐ Cada test tiene su commit **RED:** y su **GREEN:** (rojo antes que verde en el historial); a la rama solo se subió con el tope en verde.
- ☐ Los tests rotaron de autor; el trabajo está repartido entre el grupo.
- ☐ Podés explicar cada test sin mirar.

8. (Opcional, avanzado)

- **De dónde sale la palabra.** La versión base la inyecta por la URL (`?word=GAT0`) y, si no viene, usa una por defecto. Variantes permitidas: (2) que un segundo jugador escriba la palabra en una pantalla previa (suma un AT); (3) elegir una palabra al azar de una lista (necesitás un seam para inyectar el azar y poder testear determinista).
- **Capa intermedia opcional (jsdom).** Si un comportamiento de UI es difícil de cubrir con el AT, podés sumar tests de componente en jsdom (Vitest + `@testing-library/dom`). Es opcional: la fuente de verdad del "funciona" sigue siendo el AT contra la app real.

- **Tablas y Esquema del escenario en Gherkin:** para varios ejemplos del mismo comportamiento (p. ej. acertar distintas letras), usá `Scenario Outline` / `Esquema del escenario` con `Examples`, en lugar de repetir escenarios.
- **Mockear Ahorcado (escuela London/mockista):** en tests de UI más finos, reemplazar el objeto real por un mock para definir su interfaz. Más desacoplado, más complejo; el AT igual usa el objeto real.

9. Desafío: Aprobación Directa

Hasta acá tuviste una guía paso a paso. Para la **Aprobación Directa** se termina el andamiaje: el grupo elige y agrega **más funcionalidad, aplicando lo aprendido sin que nadie les marque los pasos**. Esa autonomía es lo que se evalúa.

La vara no cambia. Cada feature nueva tiene que:

- partir de una **Historia de Usuario** (nueva o de su mapa) con criterios de aceptación, trazada igual que las demás (§4.1),
- nacer de uno o más **AT en Cucumber/Gherkin** (rojo honesto primero),
- estar respaldada por **UTs sobre el dominio** (apuntá a ~100% de `src/domain/`),
- mantener la **separación lógica/UI** (la lógica nueva vive en `Ahorcado`, no en la pantalla),
- quedar registrada con **commits por transición**, y
- poder **defenderse oralmente**.

Elegí al menos cuatro de estas ideas (o proponé las tuyas):

Feature	Dónde está la lógica de dominio (lo testeable por UT)
Dibujo progresivo del ahorcado	qué partes del muñeco se muestran según la cantidad de errores (0→6)
Palabra al azar de una lista	elegir la palabra; seam para inyectar el azar y poder testear determinista
Soporte de acentos y ñ	normalizar: <code>á == a</code> , tratar la <code>ñ</code> ; conecta con "100% ≠ 0 bugs"
Teclado en pantalla	marcar las letras ya usadas (acertadas/falladas) como no disponibles
Niveles de dificultad	cantidad de vidas y/o longitud de palabra según el nivel
Pista / categoría	asociar una categoría o pista a cada palabra
Marcador de la sesión	contar partidas ganadas/perdidas en memoria
Jugar de nuevo	reiniciar el estado sin recargar la página
Dos jugadores	un jugador ingresa la palabra en una pantalla previa

Dos que recomendamos por su valor didáctico:

- **Palabra al azar** te obliga a diseñar un **seam** para el azar (igual que `?word=` lo fue para la palabra fija): es la lección de testabilidad más transferible a proyectos reales.
- **Acentos / ñ** materializa el "100% de cobertura no es 0 bugs": aparece un caso borde que ningún test previo contemplaba, y lo cazás recién cuando lo pensás.

No hay pasos dados: el grupo decide los ATs, la lista de UTs y el orden. Si en la defensa no podés explicar una decisión, no cuenta — igual que en toda la cursada.